Documentação Completa do Projeto: Análise de Tráfego com Big Data

1. Introdução ao Projeto de Big Data

Um projeto de Big Data envolve a coleta, o armazenamento, o processamento e a análise de volumes massivos de dados que sistemas tradicionais não conseguem gerenciar. Ele utiliza ferramentas modernas (como Apache Spark, Hadoop, Kafka e bancos NoSQL) para transformar dados complexos em respostas úteis.

Este documento consolida toda a especificação técnica discutida para a criação de uma infraestrutura local de análise de tráfego urbano utilizando Docker, Apache Kafka, Apache Spark, PostgreSQL e Power BI.

2. Arquitetura de Infraestrutura (docker-compose.yml)

```
version: '3.8'

services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.4.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181

  kafka:
    image: confluentinc/cp-kafka:7.4.0
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092,PLAINTEXT_INTERNAL://kafka:29092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEXT_INTERNAL:PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  spark-master:
    image: bitnami/spark:3.4.1
    ports:
      - "8080:8080"
      - "7077:7077"
    environment:
      - SPARK_MODE=master

  spark-worker:
    image: bitnami/spark:3.4.1
    depends_on:
      - spark-master
    environment:
      - SPARK_MODE=worker
      - SPARK_MASTER_URL=spark://spark-master:7077

  postgres:
    image: postgres:15
```

```
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: bigdata_user
      POSTGRES_PASSWORD: password123
      POSTGRES_DB: trafego_db
```

3. Modelo de Banco de Dados (SQL)

```
CREATE TABLE metricas_trafego (
    id_rua VARCHAR(50),
    janela_inicio TIMESTAMP,
    janela_fim TIMESTAMP,
    velocidade_media REAL,
    total_veiculos INT,
    PRIMARY KEY (id_rua, janela_inicio)
);
```

4. Simulador de Telemetria de Tráfego (Python)

Script responsável por gerar dados de GPS em tempo real de forma contínua e postar no tópico do Apache Kafka para ingestão imediata.

```
import time
import json
import random
from datetime import datetime
from confluent_kafka import Producer

config = {'bootstrap.servers': 'localhost:9092'}
producer = Producer(config)

ruas_validas = ['Av_Paulista', 'Av_Reboucas', 'Marginal_Pinheiros', 'Av_Brasil']

try:
    while True:
        dado_gps = {
            "id_veiculo": f"VEI-{random.randint(1000, 9999)}",
            "id_rua": random.choice(ruas_validas),
            "velocidade": random.randint(10, 80),
            "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        }
        producer.produce(topic='dados-trafego', value=json.dumps(dado_gps).encode('utf-8'))
        producer.flush()
        time.sleep(0.5)
except KeyboardInterrupt:
    print("Simulador encerrado.")
```

5. Processamento Streaming com PySpark

Script configurado para rodar no cluster Spark, ler o tópico do Kafka e consolidar os dados agregando-os em janelas temporais de 1 minuto antes de salvá-los no PostgreSQL.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, from_json, window, avg, count
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, TimestampType

spark = SparkSession.builder \
    .appName("ProcessamentoTrafegoTempoReal") \
    .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.12:3.4.1,org.postg
```

```
resql:postgresql:42.6.0") \
    .getOrCreate()

schema_gps = StructType([
    StructField("id_veiculo", StringType(), True),
    StructField("id_rua", StringType(), True),
    StructField("velocidade", IntegerType(), True),
    StructField("timestamp", StringType(), True)
])

df_kafka = spark.readStream.format("kafka").option("kafka.bootstrap.servers", "kafka:29092").o
ption("subscribe", "dados-trafego").load()

df_tratado = df_kafka.selectExpr("CAST(value AS STRING) as json_str") \
    .select(from_json(col("json_str"), schema_gps).alias("dados")).select("dados.*") \
    .withColumn("timestamp", col("timestamp").cast(TimestampType()))

df_metricas = df_tratado.withWatermark("timestamp", "2 minutes") \
    .groupBy(window(col("timestamp"), "1 minute"), col("id_rua")) \
    .agg(avg("velocidade").alias("velocidade_media"), count("id_veiculo").alias("total_veiculo
s")) \
    .select(col("id_rua"), col("window.start").alias("janela_inicio"), col("window.end").alias
("janela_fim"), col("velocidade_media"), col("total_veiculos"))

def gravar_no_postgres(df_lote, batch_id):
    df_lote.write.format("jdbc") \
        .option("url", "jdbc:postgresql://postgres:5432/trafego_db") \
        .option("dbtable", "metricas_trafego") \
        .option("user", "bigdata_user").option("password", "password123") \
        .option("driver", "org.postgresql.Driver").mode("append").save()

query = df_metricas.writeStream.foreachBatch(gravar_no_postgres).outputMode("update").start()
query.awaitTermination()
```

6. Configuração e Dashboards no Power BI

Para conectar o Power BI Desktop ao ambiente e obter relatórios em tempo real:

• **Modo DirectQuery:** Conecte o Power BI ao PostgreSQL (localhost:5432) usando a opção DirectQuery para garantir que as atualizações sejam feitas diretamente no banco, sem importação estática.

• **Page Refresh (Atualização Automática):** Vá no menu de Formatação da Página, ative o Page Refresh e configure o intervalo de tempo desejado (ex: 5 a 10 segundos) para automatizar as requisições visuais.